



Ben-Gurion University of the Negev

Faculty of Engineering Science

School of Electrical and Computer Engineering

Dept. of Electrical and Computer Engineering

Fourth-Year Engineering Project

Final Report

Studying the Effectiveness of Hybrid Video Compression

Project number: p-2025-109

Students: Guy Mayost-

Itamar Guerchon

Supervisors: Prof. Hadar Ofer

Mr. Dror Itai

Submitting date: 27/07/2025

חקר יעילות דחיסת וידאו היברידית לצרכי ניהוג מרחוק

שמות הסטודנטים: מיוסט גיא, גרשון איתמר

guymayo@post.bgu.ac.il

מנחים: פרופ' הדר עופר ומר. דרור איתי

התפתחות הטכנולוגיה בתחום הרכב האוטונומי מעלה את הצורך בהתערבות אנושית במקרים בהם הרכב מתקשה לקבל החלטה, ופתרון זה דורש שידור וידאו איכותי על מנת לאפשר שליטה בזמן אמת. הפרויקט עסק בייעול דחיסת וידאו עבור טלאופרטור של נהיגה מרחוק ברכבים אוטונומיים.

המטרה שלנו הייתה לאפשר לנהג המרוחק לשלוט על הרכב בעזרת איכות גבוהה של וידאו תוך הקטנת רוחב הפס הנדרש לשידור הווידאו וייעול זמן העיבוד של תהליך לכידת הפריים, יצירת תמונה היברידית ממנו, דחיסתו והעברתו למסך הטלאופרטור.

בפרויקט זה, הצענו שיטה היברידית לדחיסת וידאו, המשלבת סגמנטציה סמנטית של ה non-ROI עם שמירה על איכות גבוהה של ה ROI. לשם כך, השתמשנו בקצב ביטים נמוך של 1Mbps המדמה שימוש ברשת סלולר 4G. מאחר ואנו דימינו שידור בזמן אמת, דחיסת הווידאו הייתה בזמן אמת בעזרת NVENC עם I-Frame ו P-Frame בלבד.

השיטה בחנה באמצעות סימולטור הנהיגה שלנו, בו ייצרנו תרחישי נהיגה שונים, ובאמצעות ניסוי צופים את מידת הפרקטיות של הווידאו לנהג המרוחק. הניסוי ערך השוואה בין נסיעה עם וידאו שנדחס באופן קונבנציונלי לבין וידאו היבריד, כאשר שמרנו על רוחב פס קבוע.

התוצאות שאליהם הגענו עבור 20FPS הן זמן עיבוד ממוצע של 44.68ms עם רוחב פס ממוצע של 898kbps. זמן העיבוד כולל הוא מהרגע בו יש לנו פריים סגמנטציה סמנטית ופריים של תמונה פוטו-ראליסטית, דרך יצירת תמונה היברידית, דחיסה ופריסה והצגה על גבי המסך. המשתתפים במחקר הגיעו בצורה בטוחה ליעדם ללא התנגשות באובייקטים במרחב, אולם הם העידו כי קיים קושי בהסתגלות לנהיגה עם תמונה היברידית.

מהתוצאות שקיבלנו ותחת ההנחות שלנו, הסקנו שהשיטה ההיברידית משפרת את ביצועי הדחיסה ופריסת הווידאו בזמן אמת למסך הטלאופרטור - תוך שמירה על תפיסתו המרחבית של הנהג המרוחק. לאור

תחושות הנסיונים לעיל בתום הניסוי, אנו מציעים ניסוי צופים נוסף עם נסיונים מיומנים בנהיגה עם וידאו היברידי, כדי להעריך באופן מהימן יותר את יעילות הדחיסה ללא השפעת קשיי הסתגלות.

מילות מפתח: ROI, Latency, Bandwidth, Semantic, Segmentation, Compression, Hybrid,
Driving scenarios, Teleoperator

Studying the effectiveness of hybrid video compression

Students: Mayost Guy , Guerchon Itamar

guymayo@post.bgu.ac.il

supervisors: Prof. Hadar Ofer, Mr. Dror Itai

The development of technology in the field of autonomous vehicles raises the need for human intervention in cases where the vehicle struggles to decide. The solution requires high-quality video transmission to enable real-time control. This project deals with enhancing video compression for a teleoperator in remote driving of autonomous vehicles.

Our goal is to reduce the bandwidth required for video transmission while maintaining high image quality and constraining the processing time of frame capture, hybrid image creation, compress it, decompress it and display it on teleoperator monitor.

This project proposed a hybrid video compression method, combining semantic segmentation of non-ROI with high-quality ROI preservation. We simulated real-time 5G transmission at 1Mbps using only I-frames and P-frames. The method's practicality for remote driving was assessed via an observational experiment in our driving simulator with our various scenarios, comparing hybrid and conventionally compressed video at constant bandwidth.

The results we obtained for 20 FPS show an average processing time of 44.68 ms with an average bandwidth of 898 kbps. The total processing time is measured from the moment both the semantic segmentation frame and the photorealistic frame are available, through the creation of the hybrid image, its compression, decompression, and display on the screen. The study participants were able to safely reach their destinations without colliding with objects in the environment. However, they reported experiencing difficulty adapting to driving with hybrid imagery.

From these results, and under our stated assumptions, we concluded that the hybrid method improves real-time video compression and decompression performance for display on the teleoperator's screen, while preserving the remote driver's spatial perception. Considering the participants' feedback at the conclusion of the experiment, we recommend conducting an additional user study with participants trained to drive using hybrid video, in order to more reliably evaluate compression efficiency without the influence of adaptation challenges.

Keywords: ROI, Latency, Bandwidth, Semantic, Segmentation, Compression, Hybrid,
Driving scenarios, Teleoperator

Introduction:

Teleoperated Driving , also known as TOD, [1] is a potential solution to cope with failures in autonomous driving systems. Failure is a condition that an autonomous vehicle encounters a situation it cannot handle, such as complex urban environments, unexpected obstacles, or system malfunctions. TOD allows a remote human operator to take control. This ensures continuous operation and safety, leveraging human judgment and decision-making capabilities in scenarios where AI might struggle. By integrating TOD, the reliability of autonomous driving systems can be significantly enhanced and it provides a safety net that addresses the limitations of current AI technologies.

Displaying real-time video to teleoperators involves several challenges. One major difficulty is the need for low-latency transmission to ensure that the operator's actions are timely and effective. High latency can lead to delayed responses, which is crucial in dynamic driving environments. Moreover, maintaining high video quality is essential for surrounding awareness. However, this requires substantial bandwidth which may not always be available, often relies on cellular networks.

Our goal was to enable a remote operator to control the vehicle using high-quality video while simultaneously reducing the bandwidth required for video transmission and optimizing the processing time involved in capturing frames, generating a hybrid image, compressing it, and transmitting it to the teleoperator's display.

Different Approaches to Solving the Problem:

First approach [2] is a semantic-aware video compression , also known as SAC, framework to reduce the bandwidth of video frames from automotive cameras to the teleoperator's screen. This method involves separately compressing the region-of-interest -ROI, and the region-out-of-interest - non-ROI, within each video frame. The ROI, which includes critical elements for driving such as roads, vehicles, and pedestrians, is compressed at a lower ratio to maintain higher quality, while the non-ROI, such as sky and trees, is compressed more heavily. The slice-based compression method, SAC, offers several key advantages: it preserves high visual quality in critical driving regions, ROI, reduces the overall video file size, and improves compression efficiency without significantly affecting the performance of perception algorithms. Additionally, it enhances machine learning-based segmentation tasks, enables real-time dynamic ROI detection, provides flexibility in defining regions and compression ratios, maintains a processing time comparable to uniform compression, and integrates well with existing standards such as H.264 and H.265.

However, the method also presents certain drawbacks: visual artifacts may appear at the boundaries between ROI and Non-ROI areas, the segmentation stage introduces additional computational overhead, and further optimization is needed to improve real-time performance. Moreover, while more advanced codecs offer better compression efficiency, they remain impractical for real-time use due to their high computational demands.

The second approach [3] is a slice-based video compression method designed to reduce end-to-end latency in teleoperated driving systems. Instead of waiting for full video frames from the camera, typically captured at 60 frames per second also known as FPS, the system reads and compresses partial horizontal slices of the frame as they are received from the image sensor. Specifically, it operates at a granularity of one-sixth of a frame, corresponding to a readout time of approximately 0.0027 seconds. The compression of each slice is dynamically adjusted based on the target bitrate, which is determined by a network rate control and scheduling module. This method relies on the H.264 video compression standard due to its high compression efficiency and built-in resilience to packet loss. To support real-time streaming without waiting for lost or delayed slices, the system uses an intra refresh mode, inserting intra-predicted macroblocks into inter-predicted slices instead of transmitting full intra frames. Quantization parameters for I- and P-macroblocks are selected in real time using a lookup table derived from statistical data.

To address the H.264 encoder's requirement to buffer the entire slice before compression and the data-dependent entropy encoding time, the system uses a dual-encoder "ping-pong" architecture. While one encoder processes the current slice, the other prepares for the next, effectively doubling the slice throughput and reducing buffer memory requirements. The main advantage of this method is the significant reduction in latency, achieving end-to-end delays as low as 90 milliseconds. Additionally, the intra refresh mechanism enables fast recovery from data loss—typically within 16 frames, which is about 0.26 seconds at 60 fps, a delay short enough to remain unnoticeable to the remote driver. Real-time control of compression parameters and buffering optimization further enhance system performance. However, challenges remain: proprietary hardware was required due to the lack of commercial cameras capable of sub-frame data output, and the entropy encoding step mandates the ping-pong design. Despite its robustness, some slice loss may still occur, and there is a trade-off between recovery speed and bandwidth. Further latency improvements could be achieved by applying similar techniques on the display side.

The third approach [1] involves combining multiple camera streams into a single *superframe video*, also known as SFV, to address the bandwidth and hardware limitations commonly found in teleoperated driving systems. In such systems, the remote operator relies on real-time video streams from several vehicle-mounted cameras for situational awareness. However, many vehicles are equipped with only a single hardware video encoder due to cost and size constraints. A common workaround is to merge the frames from all cameras into a unified SFV, which is then encoded and transmitted as a single stream. This method, however, lacks the flexibility to adjust the bitrate or visual quality of each individual camera feed based on its importance in the current driving context or the available network bandwidth.

To overcome this limitation, the research proposes an innovative solution using *preprocessing filters* that allow per-camera customization of bitrate and quality while still utilizing a single hardware encoder. On the vehicle side, each camera's video stream is preprocessed individually before being combined into the SFV by a multiplexer. The combined stream is then encoded and transmitted. On the operator side, the decoded SFV is demultiplexed into separate video streams. Four types of preprocessing filters are used: first is Temporal adaptation, which reduces frame rate by skipping redundant frames. The second is Spatial adaptation, which downsamples resolution to reduce data size. The third is Quality adaptation, which applies a low-pass filter to control the amount of detail and last is Channel adaptation, which drops chrominance channels and retains only luminance to save bitrate, especially in low-light or color-insensitive scenarios. This method allows independent control over each segment's spatial, temporal, and quality parameters, achieving bitrate levels comparable to separate encoders. Notably, it reduces the total required bitrate by approximately 40% compared to conventional multiplexing without preprocessing, while maintaining the quality of the most critical views. The approach effectively treats each camera feed as an independent stream, even within a shared encoded video, much like a conductor shaping each instrument's sound individually before it reaches a single musician.

Despite its efficiency, the proposed preprocessing-based superframe approach introduces several challenges. First, there is an inherent trade-off between visual quality and bandwidth usage. Aggressive filtering can significantly reduce bitrate, but may also degrade the operator's situational awareness—especially in low-light or high-speed driving conditions. Second, the introduction of multi-dimensional adaptation mechanisms increases system complexity. The need to dynamically manage parameters such as frame rate, spatial resolution, and color channels for each camera stream requires careful coordination and real-time decision-making. Finally, the effectiveness of the preprocessing filters is highly dependent on their configuration. Filters such as low-pass or median filters can alter perceived

video quality in non-intuitive ways, potentially affecting perception metrics like VMAF or STVQM and leading to misleading quality evaluations.

Why was our approach selected:

The literature review revealed both advantages and limitations in each of the methods examined. We chose to enhance the first approach, which separates each video frame into ROI and non-ROI areas using semantic segmentation. In particular, we focus the semantic segmentation on the non-ROI areas to further reduce the amount of data transmitted per frame.

Given the constraints related to computational load, latency, and implementation complexity, we adopted the hybrid compression approach that combines photorealistic and semantic content. Our core idea is to add a single step to the conventional compression pipeline: using a pre-trained semantic segmentation model labeled with the Cityscapes dataset [\[3\]](#).

During the development phase, we utilize the CARLA simulator, which allows direct generation of semantic segmentation images from the rendering engine, without the need to run a machine learning model in real time. This is based on the assumption that in a real-world deployment, a well-trained segmentation model will be applied. Therefore, within the simulation environment, we reduce processing time and complexity while maintaining conceptual alignment with real-world conditions.

Once the segmentation is obtained, we construct a hybrid image, also referred to as an overlaid frame, in which the ROI is preserved as a photorealistic RGB image, and the non-ROI is rendered as a semantic map. This frame is compressed using the GPU's NVENC hardware encoder in H.265/HEVC format, and subsequently decoded using the NVDEC hardware decoder. The decoded video is displayed directly on-screen, allowing us to simulate a full end-to-end teleoperated driving pipeline, including image capture, compression, decoding, and visualization by the remote operator.

As part of a viewer study, we present the hybrid video to human test participants and collect feedback regarding the perceived image quality and their ability to make driving decisions in real time. This evaluation is aimed at answering the central research question: Can this method support reliable remote control of an autonomous vehicle under realistic conditions of network load, response time, and complex driving environments.

Final Technical Specification:

In this project, we conducted a viewer study to evaluate the system we developed and collect human feedback on its performance. The system processes real-time video for remote driving within the Carla simulator platform. A key design objective was to minimize bandwidth usage while maintaining high video quality in driving-relevant regions, also known as region of interest -ROI, and ensuring low latency to enable real-time vehicle control. The system consists of four main stages: frame capture, hybrid image generation, hardware-based compression, and real-time display.

1. Frame Capture:

Using Carla's Python API, we employed two virtual cameras provided by the simulator. Both cameras output video at 720p resolution means 1280×720 pixels, with a synchronized frame rate of 20–30 FPS. The first camera produces an RGB photorealistic image, while the second camera outputs a semantic segmentation image in which each pixel is classified by an object ID (e.g., road, vehicle, building), following a labeling scheme similar to Cityscapes.

2. Hybrid Image Generation:

In this stage, we classify regions into ROI and non-ROI, then combine the photorealistic image with the semantic segmentation image to generate a hybrid frame, also referred to as an overlaid frame.

3. Hardware Compression:

For compression, we utilize hardware encoders embedded in NVIDIA GPUs, specifically the NVENC component. We use NVIDIA's PyNvVideoCodec library to encode the video in H.265/HEVC format. The encoder is configured with Group of Pictures = 40, also known as GOP, meaning that for each I-frame there are 40 P-frames. This configuration improves video quality and compression efficiency.

Additionally, we limit the bandwidth to 1 Mbps.

4. Real-Time Display:

For this stage, we also use hardware components to decode the video stream encoded with NVENC. Decoding is performed using the NVDEC hardware decoder, which operates with the same NVIDIA video library.

Initially, the research proposal planned to use the GStreamer library, which supports hardware-based encoding and decoding via NVENC and NVDEC, as well as UDP-based video streaming over the internet. However, this approach would have required additional synchronization between the vehicle control interface and the video stream. Since streaming over the internet was not the project's primary goal, we decided instead to use the simpler PyNvVideoCodec library, which provides real-time compression and

decoding and allows seamless integration with PyGame for direct vehicle control. Moreover, according to the library's documentation, real-time compression benchmarks for various configurations (e.g., resolution, FPS) confirmed that PyNvVideoCodec meets our real-time requirements.

The compression format was changed from H.264 to H.265/HEVC because H.265/HEVC provides better quality at the same bandwidth, making it more suitable for our task despite its higher computational complexity. Another change from the preliminary report's planning was that we did not configure the steering wheel and pedal setup due to two major issues: a severe and long-lasting hardware failure of the laboratory computer, which left it unusable for about a month and a half, and mandatory military reserve duty during the project period. These setbacks resulted in significant time loss, so we decided to forgo configuring the steering hardware and instead control the simulator using the keyboard.

The approach taken to the solution and engineering design:

The final product is a remote-driving simulation system that replicates driving scenarios using the CARLA simulator and presents compressed overlaid frames to the teleoperator. These frames combine semantic segmentation for non-ROI with photorealistic imagery for ROI.

The core principle behind this approach is that compressing areas filled with uniform colors, which means regions that have undergone semantic segmentation, is significantly simpler and results in a substantial reduction in data size after compression. Since non-ROI regions, such as tree foliage or background buildings, do not provide critical safety information for driving decisions, they can be displayed at lower detail levels without affecting the teleoperator's situational awareness. In contrast, ROI regions are preserved at high quality to support real-time decision-making. The system is designed to maintain an end-to-end latency of approximately 200 ms, ensuring seamless and safe remote vehicle control.

Block Diagram:

The following diagram illustrates side by side conventional compressing standard video and our system. On the left side under label (a), the conventional method for compressing standard video. In this approach, frames are sent directly to the encoder and compressed without any distinction between ROI and non-ROI regions. On the right side under label (b), we present the system we developed, which introduces an additional processing step before the frame enters the encoder. In this process, the frame is divided into ROI and non-ROI regions. The non-ROI regions undergo semantic segmentation, where

each object is assigned a uniform color based on the Cityscapes labels. Afterward, the ROI and non-ROI regions are merged to form a hybrid image. This hybrid image is then passed to the encoder, resulting in a compressed frame in which the ROI automatically receives higher quality compared to the non-ROI regions.

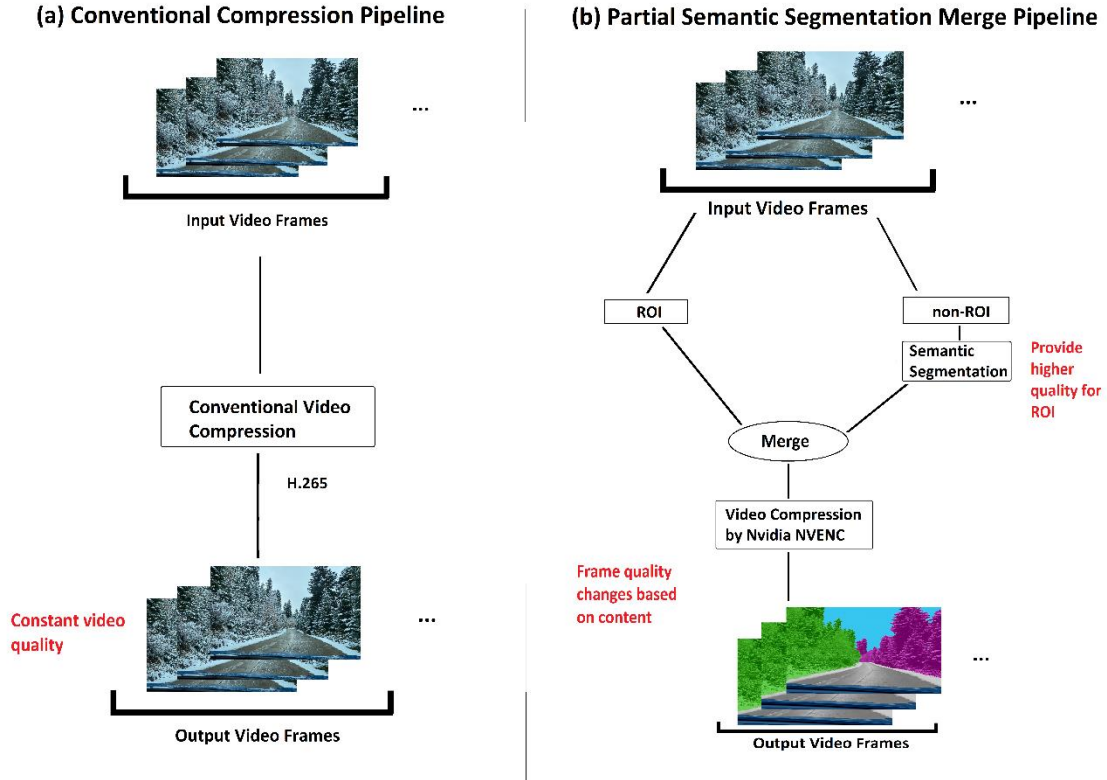


Figure. 1 -Block Diagram

System Components and Solution Approach:

1. CARLA Simulation Environment

The selected simulation environment is the CARLA simulator. This simulator provides convenient and straightforward access to its Python API, allowing full control over any parameter required. It includes realistic physics and built-in sample maps for simulation purposes. Initially, the chosen field of view was 120 degrees, located on the vehicle's roof. This choice aligns with the human visual field, which is also approximately 120 degrees. To support realistic driving scenarios, additional safety features were integrated into the vehicle, such as turn signals, front and rear lights, and more.

As stated in the introduction, this project does not involve training a semantic segmentation model. Instead, we utilize an existing pre-trained model and focus on its practical implementation. Therefore, we use the semantic segmentation generated by the simulator to combine it with the photorealistic

image and produce a hybrid image, also referred to as an overlaid frame. In this case, using semantic segmentation requires configuring an appropriate camera. We assume that the simulator has full knowledge of all objects in its map, and each object is assigned to a unique color label, similar to the Cityscapes labeling system. By assuming that all objects in the frame are already classified into their respective categories, we effectively skip the entire object detection stage.

In addition, we designed multiple driving scenarios to evaluate the test driver's performance under different conditions. Examples of these scenarios include:

1. A cyclist or pedestrian suddenly entering the roadway.
2. An ambulance approaching and entering the junction.
3. A vehicle running at a red light at the junction.
4. Avoiding an unexpected obstacle on the road.

Each scenario was implemented across multiple driving maps within the simulator.

We also developed a simulator function that logs all traffic violations committed by the test driver into a CSV file. At the end of each driving session, a CSV file is generated containing details of the violations. The traffic violation report contains the violation type, for example collision, lane crossing over a solid line, red-light running, the frame number, the timestamp of the violation, and the geographic coordinates (X, Y, Z) where it occurred. This mechanism also allows us to detect duplicate entries for the same violation.

The simulator operates at a frame rate of 20–30 FPS with a resolution of 720p, corresponding to 1280×720 pixels.

2. Merging and Creating the Hybrid Image

We capture two frames from co-located cameras: a photorealistic RGB frame and a semantic segmentation frame, similar to the Cityscapes labeling system. Cityscapes labeling system enables assigning a unique color to each object class, for example, all vehicles can be labeled in red, pedestrians in purple, and sidewalks in pink. At this stage, we work with three color channels —red expressed by R, G expressed by green, and B expressed by blue — which together form a 1×3 array, meaning one row and three columns. Each channel has values in the range of 0–255.

After defining unique color identifiers for each object class, we can apply various video-editing manipulations. We create an additional color channel for the array called alpha expressed by A, which controls transparency. The alpha channel also ranges from 0–255, where 255 represents full transparency.

In this step, as we mention, we identify the objects by their unique RGB color, we set the alpha value of the ROI to 255, making them fully transparent. This leaves the non-ROI regions in their semantic segmentation form, with the alpha channel set to 0 means fully opaque, while the ROI areas remain transparent. We then merge the photorealistic RGB frame with the processed semantic segmentation frame, resulting in a hybrid image, also referred to as an overlaid frame.

The generation of the alpha channel for each frame and the merging process are executed on the GPU using the CuPy library instead of NumPy library which operates on the CPU. This shift significantly accelerates the creation of the combined frame.



Figure. 2 - ControlLoss_9 scenario, a sudden bicycle incursion into the roadway. As we can see, camera angle set to 120 degrees, ROI such as road, sidewalk, pedestrians are photo-realistic while non-ROI such as sky, shrubs, building are semantic segmentation.

3. Compression and Transmission System

To simulate a system that compresses video in real time and transmits it over the bandwidth of a cellular network, several critical parameters must be constrained. The bitrate is limited to 1 Mbps, which corresponds to the average bandwidth of a 4G network. In addition, real-time compression involves using I-Frames and P-Frames only, excluding B-Frames because B-Frames are non-causal. As a result, video quality in real-time compression is inherently reduced.

After several tests, we determined that software-based compression using FFmpeg struggle achieve real-time encoding at 20 FPS. Moreover, the resulting video quality was unsatisfactory. Therefore, we decided to utilize hardware-based compression available on NVIDIA GPUs. Our graphics card, the NVIDIA GeForce RTX 5060, includes two hardware components: NVENC for encoding and NVDEC for decoding. (Optional: include reference to NVIDIA's documentation on encoding speed [here](#).)

We used NVIDIA's Python library PyNvVideoCodec, which provides unified access to both NVENC and NVDEC, allowing us to perform real-time compression and decompression through the same API. This simplifies integration, reduces overhead, and ensures seamless hardware acceleration throughout the entire pipeline. The selected compression format is HEVC/H.265 with GOP size of 40, meaning that for every I-Frame there are 40 P-Frames. This configuration improves video quality while reducing data size: the I-Frame is encoded independently and contains significant information, while the P-Frames are encoded relative to the preceding frame, storing only the differences. This approach reduces data volume while maintaining higher quality than using I-Frames alone.

When we send a hybrid frame to the encoder, it compresses the frame through the designated port and returns a bitstream. This bitstream is organized into Network Abstraction Layer Units, also known as NAL Units, which are the fundamental building blocks of video streams in the HEVC/H.265 standard. Each NAL Unit consists of two main parts: a NAL header followed by the payload data. The header is 16 bits long and provides essential information required for decoding and interpretation of the payload. Within this 16-bit header, the first bit is the `forbidden_zero_bit`, which must always be set to zero and indicates a stream error if set to one. The following six bits represent the `nal_unit_type`, which defines the type of information carried by the NAL Unit and can include parameter sets such as VPS - Video Parameter Set, SPS - Sequence Parameter Set, PPS - Picture Parameter Set, as well as coded video slices like I-frames and P-frames and auxiliary data such as SEI - Supplemental Enhancement Information. The next six bits are the `nuh_layer_id`, used to identify the coding layer in scalable or layered video coding scenarios, and the last three bits form the `nuh_temporal_id_plus1` field, which indicates the temporal layer for temporal scalability and is always greater than zero. After the header comes the payload, which contains the actual encoded data such as parameter sets, coded frame data, or supplemental metadata. This structure allows the decoder to parse the bitstream efficiently and determine how each unit should be processed. When receiving the bitstream from the encoder, the system must parse and separate each of these components before forwarding them individually to the decoder.

4. Video Display

After NVDEC completes the decoding of the video stream, it produces a fully reconstructed frame that can be displayed on the screen. We chose to transfer the decoded frame back to the CPU and render it using a PyGame display rather than displaying it directly from the GPU. This decision was made because we required seamless vehicle control through the keyboard. Rendering via the CPU allowed us to synchronize the video display with keyboard interrupts more efficiently. By returning the frame to the CPU for display in PyGame, we avoided the overhead of GPU-based rendering and achieved smoother integration between video playback and control input.

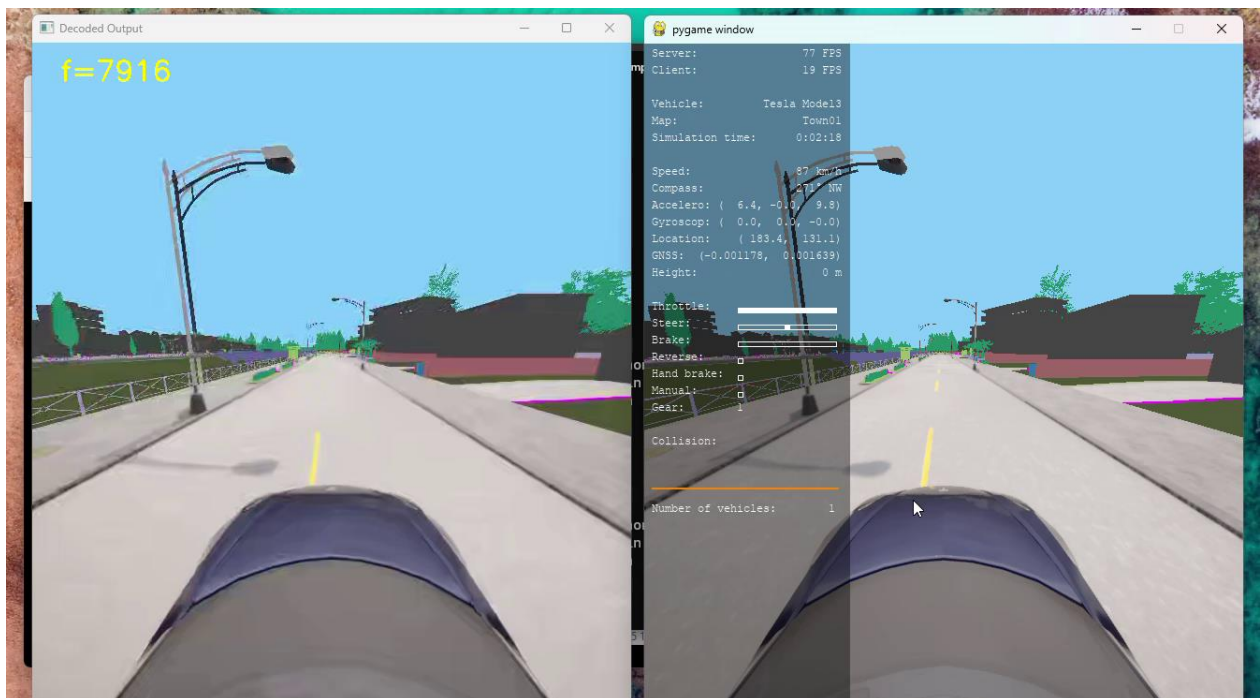


Figure. 3 – FreeRide_3 scenario, Side by side live stream decoded video and raw video. On the left side the compressed and decoded video and on the right side the raw video unencoded.

5. Hardware

The system was built on a computer equipped with an Intel i7-12700F CPU and 32 GB of DDR5 RAM. To support both the execution of the CARLA simulator and hardware-accelerated video processing, an NVIDIA RTX-5060 GPU was added, utilizing its NVENC component for encoding and NVDEC component for decoding. A Logitech G920 steering wheel and pedal set was also included for driving simulation, although we were unable to fully configure it within the project's timeframe. The system output was displayed on a 720p monitor.

6. Viewer Experiment

The objective of the viewer experiment was to evaluate the impact of displaying compressed hybrid video — which preserves high quality in ROI while simplifying less relevant areas, non-ROI — on drivers' real-time decision-making and overall driving performance. Unlike previous studies that we discuss on preliminary report, focus on quantitative metrics such as PSNR or MSE, this experiment emphasizes the human operator's experience, reaction to critical events, and subjective assessment of display quality. During the familiarization phase, the driver was given approximately five minutes of free driving without pedestrian or traffic interactions to adapt to the 2D driving environment and the hybrid video overlay. After this phase, the driver was presented with five scenarios simulating common but challenging driving situations:

1. ChangeLane_1_Long – lane change on an intercity road.
2. ChangeLane_1_Short – lane change on an urban road.
3. ControlLoss_9 – sudden bicycle incursion into the roadway.
4. FollowLeadingVehicleWithObstacle_11 – following a leading vehicle while avoiding a road obstacle.
5. OppositeVehicleRunningRedLight_2 – an oncoming vehicle running a red light at an intersection.

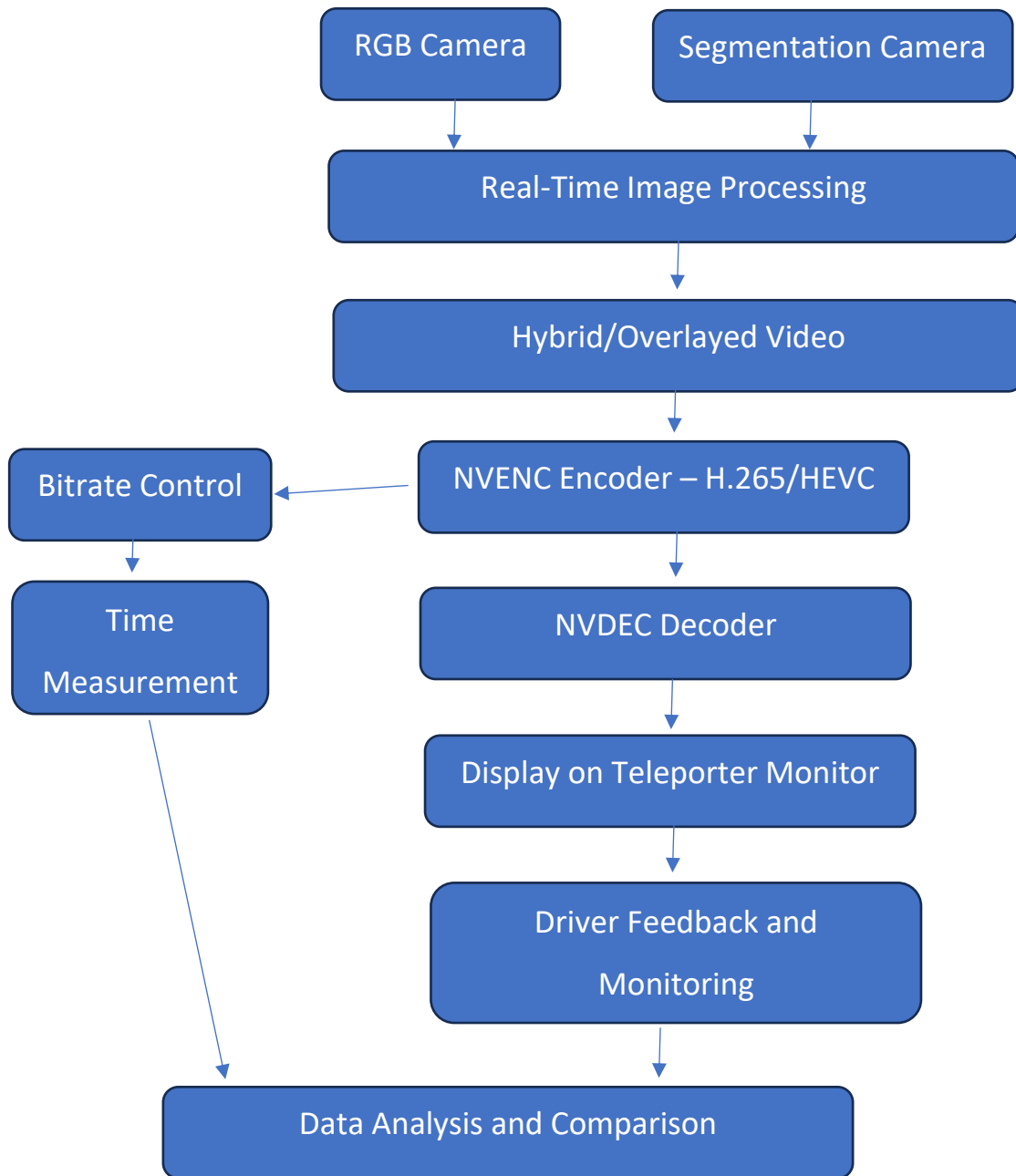
Following each scenario, collision data were automatically recorded from the simulator, along with qualitative driver feedback regarding their experience and perceived information clarity in the display.



Figure. 4 – OppositeVehicleRunningRedLight_2. Both cars have green light.

Design Considerations:

The driving scenarios were selected to be simple yet representative, enabling a focused evaluation of the system's effectiveness in critical road conditions. The familiarization phase was designed to minimize learning-curve effects on driver performance and to provide a consistent baseline among participants. Furthermore, analyzing driver reactions to high-quality ROI versus simplified Non-ROI regions provides insights into how the hybrid video presentation influences performance. The combination of objective data (such as collisions and response times) with subjective feedback (driver impressions) enables a comprehensive assessment of the system's effectiveness.

System diagram:

Final acceptance tests for the code:

In our literature review [4], we found that the bandwidth required for good-quality video calls is at least 1.5 Mbps for one-on-one communication. Considering this limitation, we set a target to reduce the bandwidth by approximately 30% while maintaining sufficient image quality to allow remote vehicle control. To evaluate whether this target was achieved, we developed a function that records the actual bitrate every 0.5 seconds into a CSV file. At the end of each driving session, the collected data was processed into a graph showing bitrate variation over time.

The bitrate calculation was performed by the size of the current packet that we received out of the encoder multiply by 8 to get bits units and dividing the total time that took the encoder to encode the frame, according to the following formula:

$$bitrate = \frac{Bytes\ of\ the\ current\ packet * 8}{\Delta t}$$

The resulting graph shows that the average bitrate was approximately 898 kbps, which is very close to the defined target of 1 Mbps. This indicates successful adherence to the bandwidth constraints set for the system. Most measurements demonstrate a relatively stable bitrate ranging between 800 and 1000 kbps, with periodic fluctuations characteristic of the GOP structure, where I-frames consume more bits than P-frames. A few isolated deviations can be observed at the start of the session and during certain intervals, likely caused by transitions between GOP groups or synchronization events. Additionally, the analysis revealed that the decoder becomes fully operational only after the first three to four frames, explaining the sharp drop in bitrate observed at the beginning of the graph before stabilization occurs. Overall, the graph demonstrates that the system maintains stable video quality while meeting the defined bandwidth constraints, thereby validating its suitability for real-time remote driving applications.

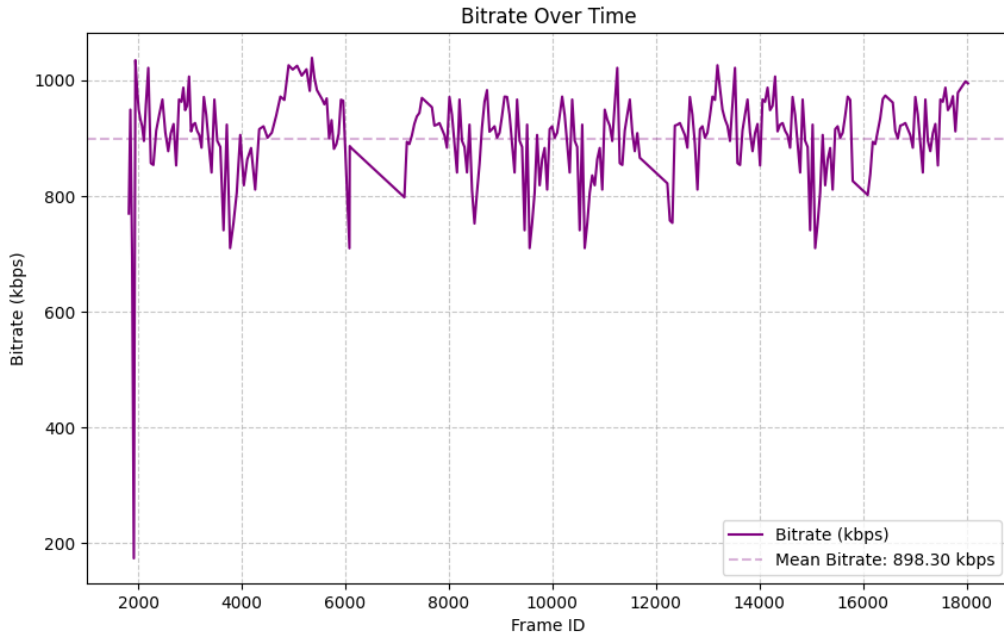


Figure. 5 – Bitrate over time.

Another target we set for this project was to achieve a total processing time of no more than 200 ms from the moment a frame is captured until it is displayed on the teleoperator's screen, a metric commonly referred to as glass-to-glass latency or end-to-end latency. In our literature review [4], we found that network latency constitutes a major portion of the total end-to-end latency. Specifically, the average network latency measured for the transmission of a full video frame is 96.1 ms. Network latency is defined as the time elapsed between sending a packet and receiving the corresponding acknowledgment at the application layer on the client side.

Based on these findings, and given that we have no control over the cellular network to reduce network latency, we allocate approximately 100 ms to complete all image processing tasks on our side.

Furthermore, in the literature review we referenced the paper *Efficient Semantic Segmentation for Compressed Video* [5], in which the researchers developed a model called **WTDecomNet**. This model can perform semantic segmentation at a rate of 27 FPS on the Cityscapes dataset, achieving about a 70% reduction in computational cost compared to baseline methods using a single NVIDIA GeForce RTX 3090 GPU. We assume that the semantic segmentation process operates on a dedicated GPU, separate from

the GPU used for our video processing pipeline. Therefore, for a single frame, the semantic segmentation processing time can be estimated as:

$$\text{Segmentation Process Time} = \frac{1}{27} = 37.04\text{ms}.$$

Therefore, the remaining time budget for the rest of the processing pipeline is approximately **63 ms**. To analyze this, we conducted the following measurements:

1. From the creation of the frame until the completion of the merging process that produces the hybrid image.
2. The encoding time using the NVENC hardware encoder.
3. The decoding time using the NVDEC hardware decoder and the subsequent display on the screen.
4. The total time from frame creation to display on the teleoperator's screen.

1. From frame creation to completion of the hybrid image merging process

Frame creation simulates frame capture by camera's lens and the result is an array. First, we capture the RGB frame and subsequently capture the semantic segmentation frame. In the next step, using the GPU and the CuPy library, for parallel process, we add an alpha channel to the semantic segmentation frame. We then set the ROI regions to full transparency, leaving only the non-ROI regions rendered in uniform colors corresponding to each object. Following this, the processed semantic segmentation frame is merged with the RGB frame on the GPU, again using CuPy. We measured the duration of this entire process and logged it into a CSV file. During analysis, we applied a moving average over the last 100 samples to smooth the results. On the next figure, we can see the latency per frame and moving average of last 100 samples. The measurements show that the average time required to generate the hybrid image, also referred to as the overlaid frame, is below 10 ms.

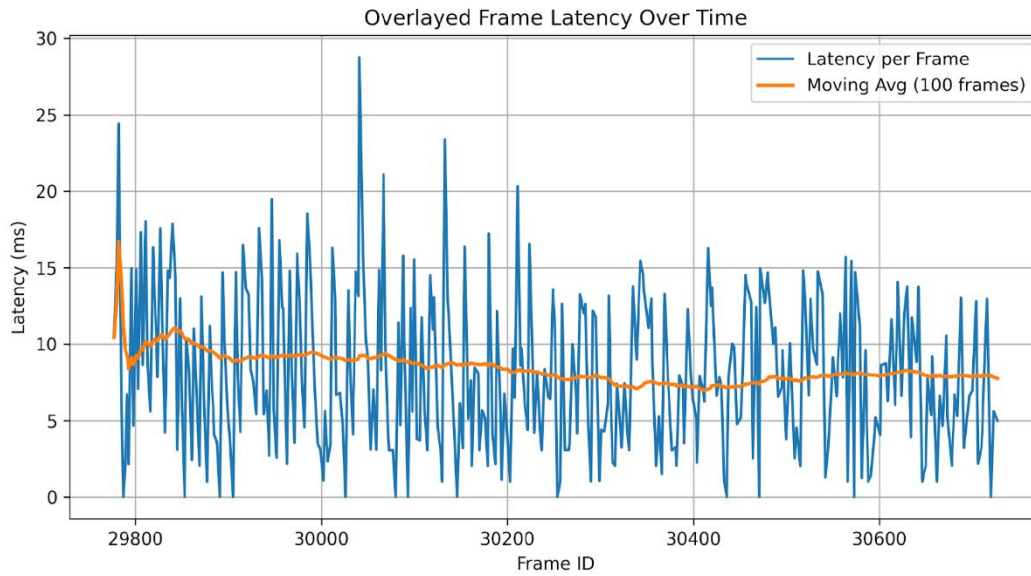


Figure. 6 – Latency per frame of the hybrid frame process.

2. Hardware encoding time

The hardware encoding time was measured from the moment the hybrid frame was submitted to the encoder until the last bit of the resulting bitstream was received. This bitstream was then encapsulated into a packet. As mentioned earlier, the encoder processes the frame and produces a stream of bits that includes additional overhead associated with the HEVC/H.265 compression format.

3. Hardware decoding and display time

The hardware decoding time was measured from the moment the packet was fully formed, through the parsing of the headers, as described in detail earlier, until a fully decoded frame was obtained. This frame, reconstructed according to the I-frame within the GOP structure, was then displayed on the teleoperator's screen.

4. Total end-to-end processing time

Total processing time was measured from the capture of the frame until its display on the screen. We measured the whole process separately due to communication time between the GPU, CPU and other peripheral components in the computer. As we can see in the next graph that shows encoding time in blue line, decoding time and display on screen in orange line, total process time in green and the average time of the whole process in red dashed line. We can see that the total process time is greater than sum of encoding time, decoding time and display. The result shows that average total time is 44.68 ms for the entire pipeline.

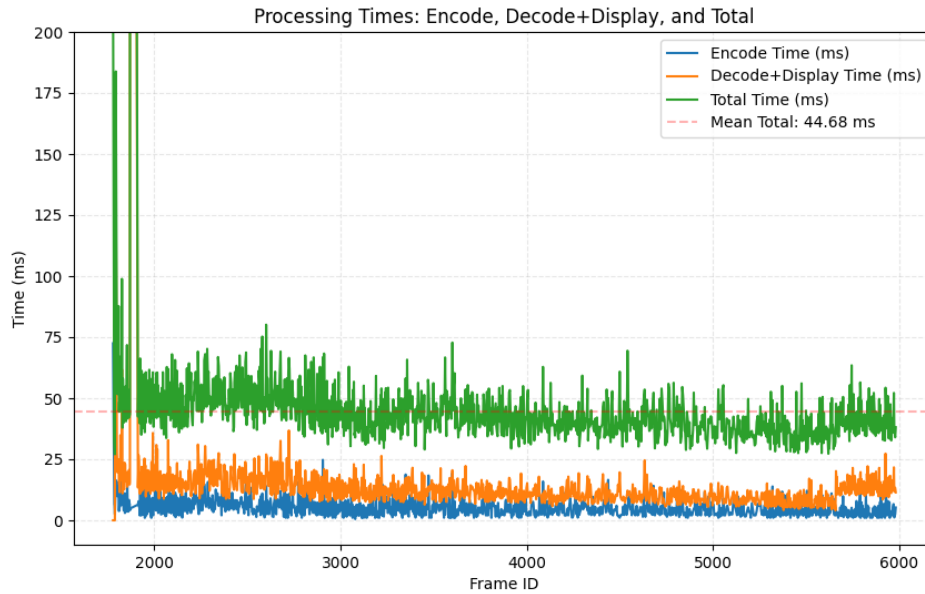


Figure. 7 – Encoding time, Decoding and display time, and total time.

The measurements were conducted over 4,000 frames. At the beginning of the measurements, a noticeable peak in processing time was observed and after short time the system stabilized and remained consistent throughout the remainder of the process.

By combining all the timing components discussed in this section, and assuming a network latency of approximately 100 ms, along with an estimated semantic segmentation processing time of 37 ms and our measured end-to-end processing time is about 45 ms, which includes frame capture, hybrid frame generation, hardware encoding, hardware decoding, and display. We arrive at an average total of approximately 182 ms for the entire pipeline. This result demonstrates that our system successfully meets the design goal of remaining below the 200 ms glass-to-glass latency target established for the project.

During the viewer experiment, we evaluated three different test drivers in the simulator. Due to a tight schedule resulting from mandatory military reserve duty and the laboratory computer being out of service for approximately a month and a half, the number of participants was limited. Each participant was first given five minutes of free driving in a scenario called FreeRide_3 to familiarize themselves with the environment, followed by five different test scenarios:

1. ChangeLane_1_Long – lane change on an intercity road.

2. ChangeLane_1_Short – lane change on an urban road.
3. ControlLoss_9 – a sudden bicycle incursion into the roadway.
4. FollowLeadingVehicleWithObstacle_11 – following a leading vehicle while avoiding obstacles placed on the road.
5. OppositeVehicleRunningRedLight_2 – an oncoming vehicle running a red light at an intersection.



Figure. 8 – ChangeLane_1_Long – lane change on an intercity road.

These scenarios were relatively basic and designed primarily to test the driver's ability to interpret and react to the driving environment. Participants were instructed to drive at speed they felt comfortable with operating the vehicle. None of the test drivers collided with any objects during the scenarios. However, deviations from the driving lane were observed. Such deviations can be attributed to two possible causes: first, a lack of familiarity with the vehicle's dimensions and second, the use of keyboard controls instead of a steering wheel. Keyboard inputs are binary, meaning acceleration is either fully applied or not applied at all, and therefore lack the sensitivity and fine control provided by a dedicated steering wheel and pedal setup.

Challenges and Their Resolutions

Throughout the project, we encountered several major challenges that required both technical and logistical solutions.

The first challenge arose during the creation of the hybrid image. This occurred in the early stages of exploring the simulator to understand the available tools and capabilities. At this stage, we were also beginning to develop the project and gain a deeper understanding of video compression, video structure, and related processes. Once we acquired sufficient knowledge in both video processing and simulator operation, we discovered the use of the alpha channel, which controls pixel transparency. This insight enabled us to make the ROI in the semantic image fully transparent while keeping the non-ROI areas fully opaque. Initially, the image processing was handled via the NumPy library, which operates sequentially on the CPU and was therefore too slow for real-time processing. The solution was to transition to the CuPy library, which performs parallel processing on the GPU and significantly accelerated hybrid frame creation.

The second challenge involved video compression. We discovered that software-based compression struggled to maintain real-time performance, even at 20 FPS, and the resulting image quality was insufficient. Attempts to improve image quality only created bottlenecks that negatively affected the entire system's performance. The solution was to utilize hardware-based compression, specifically the NVENC and NVDEC components of the NVIDIA GPU. These hardware encoders and decoders operate independently of the GPU's main compute cores, allowing us to achieve real-time performance without impacting other system processes.

Another significant challenge was encountered during the decoding process. While software-based compression using FFmpeg was straightforward and intuitive, transitioning to NVENC for encoding and NVDEC for decoding required manual management of frame timing, bitstream handling, buffer flushing, and simulator integration. To address this, we acquired a deep understanding of the H.265/HEVC compression format, which enabled us to develop functions to properly manage the encoding and decoding pipeline.

In addition to these technical challenges, we also faced hardware failures. The laboratory computer used to run the project failed for an extended period of approximately six weeks. During this time, we purchased a personal computer with lower processing capabilities due to budget constraints and continued development on this interim setup.

At the outset of the project, it was a collaborative effort between Ben-Gurion University and Elbit Systems. The company required the use of Microsoft's AirSim simulator, which had become outdated

and was no longer supported. Working with AirSim presented significant difficulties due to compatibility issues with newer hardware and outdated libraries. Midway through the project, Elbit Systems discontinued its collaboration, leading us to switch to a newer and more widely supported simulator, CARLA, which offered clear programming advantages. However, working with two simulators consumed considerable development time.

Finally, due to the war our country is facing, we encountered additional challenges. During the project, I was called up for mandatory military reserve duty, which removed me from active development for approximately six weeks.

Conclusions and recommendations:

The platform developed in this project enables the evaluation and training of drivers to examine the feasibility of operating a hybrid-image-based system for remote driving. We demonstrated that it is possible to drive in a simulator using hybrid imagery while achieving approximately 30% bandwidth savings compared to standard video streaming. In the future, when a prototype vehicle based on this approach is constructed, the platform we created can be utilized for driver training, thereby reducing both costs and the risks associated with physical training. Furthermore, the system's total end-to-end processing time averaged 182 milliseconds, which is below the 200-millisecond target defined at the outset of the project — a result that meets critical requirements for real-time remote driving.

Alongside these advantages, several key limitations were identified. The most significant drawback, as reported by the test participants, was the difficulty in adapting to the hybrid image, the vehicle's dimensions, and two-dimensional teleoperation, which affected driving accuracy and vehicle control during the trials. For future studies, it is recommended to provide participants with extended acclimatization and training sessions before commencing formal measurements. In addition, this project did not evaluate the combined effect of the semantic segmentation model with the video processing and compression pipeline, in part due to computational constraints imposed by the CARLA simulator. These limitations hinder the ability to assess the overall performance of such a system on a single GPU and highlight a challenge that should be addressed in future system development and in the transition to real-world vehicle testing.

Planning versus Execution:

In the initial planning stage, as presented in the preliminary report, we defined the following goals and methodologies:

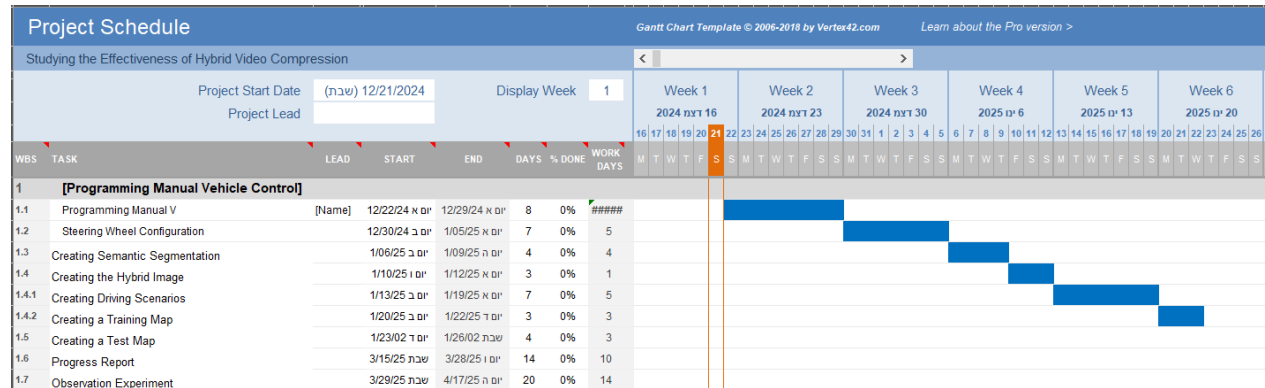
1. Use the Carla simulator to create diverse driving scenarios under different weather conditions, such as sunny, rainy, and cloudy, combining both urban and interurban environments.
2. Implement hybrid video using a filter that identifies non-ROI objects and colors them uniformly according to Cityscapes labels.
3. Use GStreamer to compress video in H.264 format and display the compressed video on screen in real time.
4. Use a Logitech G29 steering wheel and pedal set for the viewer experiment to ensure a realistic driving experience.
5. Conduct performance measurements including bandwidth, latency, and driving accuracy during the viewer experiments.

Most of these core design principles were maintained throughout the project; however, several adjustments and changes were implemented:

1. Although we temporarily worked with another simulator, AirSim by Microsoft, due to a requirement introduced after the preliminary report from Elbit Systems (with whom we were collaborating), we ultimately continued to work with Carla as originally planned.
2. The process of creating the hybrid image was carried out as initially designed.
3. Instead of using GStreamer, which proved complicated to install and maintain and posed challenges with version updates, we decided to work with NVIDIA's Python library PyNvVideoCodec for compression and decompression. This allowed us to utilize the NVENC hardware component for encoding and NVDEC for decoding. Additionally, we upgraded the compression format from H.264 to the more advanced H.265/HEVC to achieve greater compression efficiency.
4. The steering wheel and pedal set purchased was the Logitech G920, chosen during the period when we worked with AirSim. However, due to an extended hardware failure of the lab computer and time constraints caused by reserve military duty, we decided to forego the wheel and pedals altogether. As a result, the viewer experiment was conducted using keyboard controls, which are binary and lack sensitivity to input pressure, leading to lane deviations among participants.
5. We conducted end-to-end measurements of bandwidth and latency and confirmed that the project achieved its defined targets: a 30% reduction in bandwidth and an average processing time of 182 ms compared to the 200-ms target set during the planning phase.

Punctuality:

The following Gantt chart is taken from the preliminary report and reflects the original project plan prior to the adjustments made during execution:

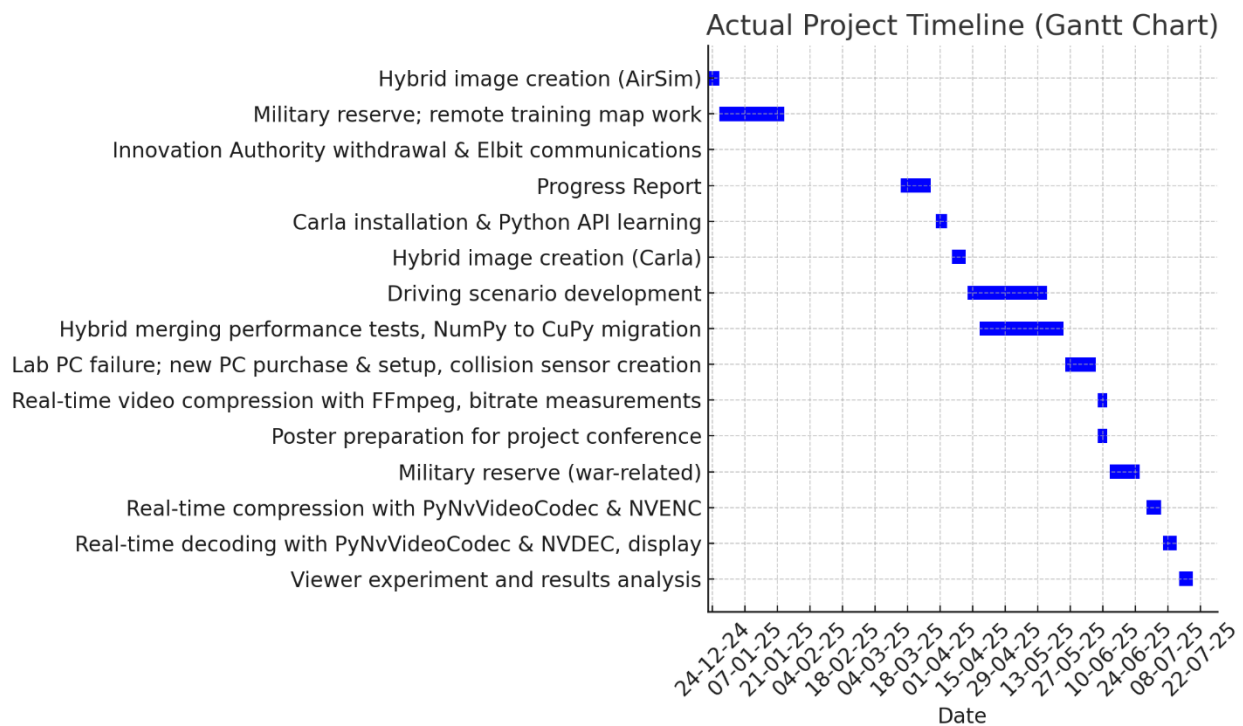


What Actually Happened:

We decided to change the order of work and prioritize tasks based on their level of importance. As a result, work on the steering wheel and pedal configuration was not performed at the beginning, and moreover, the steering wheel and pedals only arrived at the beginning of March. Initially, we worked with Microsoft's AirSim simulator. Over the course of the project, numerous changes occurred due to the transition between simulators, mandatory military reserve duty, and a failure of the laboratory computer, all of which affected the project timeline. The following is a detailed description of the work that was actually carried out:

Start Date	End Date	Duration (days)	Task / Activity	Notes
22/12/2024	27/12/2024	6	Hybrid image creation in AirSim environment	
27/12/2024	24/01/2025	29	Military reserve, remote work on training map for simulation	
05/02/2025	14/03/2025	1	Letter from Innovation Authority ending support communications with Elbit Systems	
15/03/2025	28/03/2025	14	Work on Progress Report as originally planned	
30/03/2025	04/04/2025	6	Installation of Carla simulator and learning Python API	
06/04/2025	12/04/2025	7	Hybrid image creation in Carla	
13/04/2025	17/05/2025	35	Development of driving scenarios in simulator	
18/04/2025	24/05/2025	37	Testing hybrid merging performance, migration from NumPy to CuPy for acceleration	
25/05/2025	07/06/2025	14	Major lab PC failure, purchase of new PC, reinstallation, and creation of collision detection sensor	Lab PC failure caused delays
08/06/2025	12/06/2025	5	Real-time video compression with FFmpeg, bitrate measurements, decision to move to another solution	
08/06/2025	12/06/2025	5	Parallel work on poster for project conference	
13/06/2025	26/06/2025	14	Additional military reserve due to "Am Kalavi" war	External factor (war)
29/06/2025	05/07/2025	7	Real-time compression using PyNvVideoCodec and NVENC hardware	
06/07/2025	12/07/2025	7	Real-time decoding using PyNvVideoCodec and NVDEC, display on screen	
13/07/2025	19/07/2025	7	Full viewer experiment and results analysis	

We created a Gantt chart that illustrates our progress throughout the year:



Costs:

The following table is taken from the preliminary report and presents the initially estimated costs of the equipment we expected to purchase:

Equipment	Price	Quantity
Logitech G29	1500 NIS	1
Monitor	1000 NIS	3
Total	4500 NIS	

In practice, due to a hardware failure of the laboratory computer, we were highly motivated to achieve the project's goals and therefore purchased a new computer at our own expense. Since several monitors were already available in the laboratory, there was no need for an additional purchase. The actual cost table is as follows:

Equipment	Price	Quantity
Logitech G920 (Steering Wheel + Pedals)	1078 NIS	1
PC with suitable GPU	4500 NIS	1
Total	5578 NIS	

In the appendices, an email correspondence from the supervisor, Mr. Itai Dror, is attached regarding the order of the steering wheel and pedals.

Appendices:

דואר נכנס x

לא בדומיין

Fwd: הזמנת הגה ודבשות לצרכי מחקר עם אלביט



24 ביולי 2025, 10:54 (לפני יומיים)

Itai Dror

אני ▾



----- Forwarded message -----

From: Itai Dror <itaidror1@gmail.com>

Date: Thu, Feb 13, 2025 at 2:30 PM

Subject: הזמנת הגה ודבשות לצרכי מחקר עם אלביט

To: <engres@bgu.ac.il>, עופר הדר, <ofer.hadar2@gmail.com>

לאה שלום,

אבקש את ההגה + דושות הבא:

<https://www.gamemobile.co.il/items/2633244-Logitech-Driving-Force-G920>

הספק הינו ספק של האוניברסיטה.

יש להוסיף 49 ש"ח עבור המשלוח.

המחיר כולל המשלוח וכולל מע"מ הינו 1078 ש"ח.

דיברתי עם החנות ויש במלאי.

בברכה

איתי דרור

054-9222383

References:

[1] M. Hofbauer, C. B. Kuhn, G. Petrovic, and E. Steinbach, "Adaptive Multi-View Live Video Streaming for Teledriving Using a Single Hardware Encoder," in *Proc. 2020 IEEE International Symposium on Multimedia (ISM)*, Dec. 2020, pp. 9-16.

[2] Y. Wang, P. H. Chan and V. Donzella, "Semantic-Aware Video Compression for Automotive Cameras," in *IEEE Transactions on Intelligent Vehicles*, vol. 8, no. 6, pp. 3712-3722, June 2023.

[3] Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., Franke, U., Roth, S., & Schiele, B. (2016). "The Cityscapes Dataset for Semantic Urban Scene Understanding". In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 3213-3223).

[4] A. Belogolovy, D. Dasalukunte, R. Dorrance, E. Stupachenko, and X. Zhang, "Low latency communication over commercially available LTE and remote driving," *arXiv preprint arXiv:2209.09794*, pp. 1-7, Sep. 2022.

[5] Z. Zhang, S. Shi, V. Gupta, and R. Jana, "Analysis of Cellular Network Latency for Edge-Based Remote Rendering Streaming Applications," in *Proc. ACM SIGCOMM Workshop on Networking for Emerging Applications and Technologies (NEAT'19)*, Beijing, China, Aug. 2019, pp. 1–7.

[5] J. Cai, Q. Li, Y. Shen, J. Pan and W. Liu, "Efficient Semantic Segmentation for Compressed Video," *2024 IEEE International Conference on Robotics and Automation (ICRA)*, Yokohama, Japan, 2024, pp. 4266-4272.

המלצת ציון לדו"ח מסכם

אם יש צורך, לכל סטודנט/ית בנפרד

מספר הפרויקט: 2025-109-p

שם הפרויקט: חקר יעילות דחיסת ווידאו היברידית לצרכי ניהוג מרחוק

שם המנחה מהמחלקה: פרופסור עופר הדר

מר איתי דרור

שם הסטודנטים: איתמר גרשון

ת.ז. -

גיא מיוסט

ת.ז. -

קריטריון	%	1 - חלש	2 - בינוני	3 - טוב	4 - ט"מ	5 - מצוין
הגדרת המטרה - האם מטרת הפרויקט ברורה? המטרה צריכה לכלול גם את התרומה הצפויה מהשלמת הפרויקט.	14					
ארכיטקטורת הפתרון - האם בוצעה סקירה טובה של הנושא? האם הובהר מהי הארכיטקטורה המוצעת לפתרון ומדוע היא מתאימה לפתרון הבעיה? האם הייתה התייחסות לאלטרנטיבות?	14					
הצגת התוצאות ראשוניות - האם הוסברו המשמעויות של התוצאות (החלקיות), או המסקנות הנובעות מהן, האם רק היתה הצגה עובדתית של תוצאות?	14					
ההצגה עצמה - האם יש מבנה הגיוני להצגה? מבוא, רקע ו-SoTA, מטרות, שיטה, פתרון, תוצאות, מסקנות.	15					
רמת קושי - כיצד הנך מעריך את עבודת הסטודנט על הפרויקט עד כה, ביחס לסטודנט *ממוצע* בקורסים אותם אתה מלמד?	14					
הבנה ועצמאות - האם נראה שהסטודנט מבין את העבודה שעשו, האם המענה לשאלות מספק? האם זו הייתה עבודה עצמאית?	15					
שורה תחתונה - כיצד הנך מעריך את התקדמות הפרויקט ביחס לכלל הפרויקטים שראית?	14					

הערכת רמת הקושי של הפרויקט: קל מאוד / קל / בינוני / קשה / קשה מאוד הערות:

הערות:

12.2 טופס כיבוד זכויות יוצרים וסודיות

יש לצרף את הטופס הבא חתום לדוח מכין:

אני מצהיר שלא אעשה שימוש בפרויקט ההנדסי שלי בכל חומר בעל זכויות יוצרים כגון:

טקסט,

תמונה,

אודיו,

וידאו,

מוזיקה,

סרט,

אנימציה,

תוכנה

חומרה

תיכון מעגל

ללא קבלת אישור מראש מבעל הזכויות

אני מצהיר שאשלב בפרויקט ההנדסי שלי בוחות, סרטונים, והרצאות אינפורמציה שאינה נחלת הכלל

רק בתנאי שאושרה מראש ע"י בעל הזכויות.

הרישום לפרויקט ההנדסי משמש ההתחייבות שלי לקיים ולכבד זכויות יוצרים וסודיות

חתימה: א'תמר ארז תאריך: 16.11.24

א'א' ניסן